# AWS SAA-C03 Study Notes: Decoupling Applications

## 1. Introduction to Messaging

- **Synchronous Communication:** Application components talk directly to each other (e.g., Service A calls Service B). If Service B fails or slows down, Service A fails or slows down.
- **Asynchronous Communication (Decoupling):** A buffer (queue or topic) is placed between components.
  - Service A (Producer) sends a message to the buffer.
  - Service B (Consumer) reads from the buffer when ready.
- **Benefits:**
  - **Scalability:** Components scale independently.
  - **Resiliency:** If the consumer is down, messages are stored, not lost.

## 2. Amazon SQS - Standard Queues Overview

- **SQS (Simple Queue Service):** Fully managed message queuing service. Pull-based.
- **Standard Queue Attributes:**
  - **Unlimited Throughput:** Supports nearly unlimited API calls per second.
  - **At-Least-Once Delivery:** Messages are guaranteed to be delivered, but **duplicates** may occur.
  - **Best-Effort Ordering:** Messages are generally delivered in order, but exact order is **not** guaranteed.
- **Data Retention:** Default 4 days, maximum 14 days.
- **Message Size:** Max 256KB (Use SQS Extended Client library + S3 for larger messages).

## 3. SQS - Message Visibility Timeout

- **Concept:** When a consumer polls a message, it becomes "invisible" to other consumers so it isn't processed twice simultaneously. It is *not* deleted yet.
- **Default Timeout:** 30 seconds.
- **Scenario:** If processing takes longer than the timeout, the message reappears in the queue and is processed again (duplicate processing).
- **Solution:** The consumer must call the `ChangeMessageVisibility` API to get more time if processing is heavy.
- **Dead Letter Queue (DLQ):** If a message fails processing multiple times (max receives threshold), it is sent to a DLQ for debugging.

## 4. SQS - Long Polling

- **Short Polling (Default):** Returns immediately, even if the queue is empty (returns distinct empty response). Can be expensive due to API costs.
- **Long Polling:** The consumer waits (up to 20 seconds) for a message to arrive in the queue.
- **Benefits:**
  - Decreases the number of empty responses.
  - Reduces latency (message is sent to consumer immediately upon arrival).
  - **Reduces Cost** (fewer API calls).
- **Configuration:** Can be set at the Queue level ( `ReceiveMessageWaitTimeSeconds` ) or the API call level ( `WaitTimeSeconds` ).

## 5. SQS - FIFO Queues (First-In-First-Out)

- **Use Case:** When order is critical and duplicates are not allowed (e.g., banking transactions).
- **Key Features:**
  - **First-In-First-Out Delivery:** Strictly preserves message order.
  - **Exactly-Once Processing:** No duplicates.
  - **Limited Throughput:** 300 msg/s without batching; 3,000 msg/s with batching.
- **Deduplication ID:** Used to prevent duplicate messages sent by the producer.
- **Message Group ID:** Allows parallel processing of different groups (e.g., Bus ID 1, Bus ID 2) while maintaining order *within* the group.

## 6. SQS + Auto Scaling Group (ASG)

- **Pattern:** Elastic scaling based on load.
- **Metric:** CloudWatch Metric `ApproximateNumberOfMessagesVisible` .
- **Workflow:**
  1. Queue length increases.
  2. CloudWatch Alarm triggers.
  3. ASG Scale-Out Policy adds EC2 instances to process messages faster.
- **Target Tracking:** The most effective scaling policy (e.g., "Keep backlog per instance at 1000").

## 7. Amazon Simple Notification Service (AWS SNS)

- **Pattern:** Pub/Sub (Publish/Subscribe). Push-based.
- **Producer:** Sends one message to an SNS **Topic**.
- **Subscribers:** Can be SQS, Lambda, HTTP/S endpoint, Email, SMS, Mobile Push.
- **Key Features:**
  - Instant push delivery (no polling).

- **Message Filtering:** JSON policy allows subscribers to only receive a subset of messages.
- **Encryption:** Standard encryption in transit and at rest.

## 8. SNS and SQS - Fan Out Pattern

- **Problem:** You want to send one message (e.g., "New Order Placed") but trigger multiple independent actions (Email User, Update Inventory, Notify Fraud Service).
- **Solution:**
  1. Create an SNS Topic.
  2. Create multiple SQS Queues (one for Inventory, one for Fraud, etc.).
  3. **Subscribe** the SQS Queues to the SNS Topic.
  4. Publish the message *once* to SNS.
- **Result:** SNS pushes the message to all SQS queues in parallel. The services process from their own queues at their own pace. ensures **data persistence** and **decoupling**.

## 9. Amazon Kinesis Data Streams

- **Purpose:** Ingest real-time streaming data at scale (logs, metrics, IoT, clickstreams) for Big Data analytics.
- **Architecture:**
  - **Shards:** Streams are divided into Shards. 1 Shard = 1MB/s write or 1000 records/s.
  - **Retention:** Default 24 hours, up to 365 days. (Unlike SQS, data is *not* deleted after consumption; it expires).
  - **Ordering:** Ordered *per shard*.
- **Capacity Modes:**
  - **Provisioned:** You choose number of shards manually.
  - **On-Demand:** AWS scales shards automatically.

## 10. Amazon Data Firehose (formerly Kinesis Firehose)

- **Purpose:** Fully managed service to **load** streaming data into destinations.
- **Destinations:** Amazon S3, Redshift, OpenSearch, Splunk, HTTP Endpoint.
- **Nature:** Near real-time (lowest latency is 60 seconds).
- **Features:**
  - Auto-scaling (no shards to manage).
  - Data Transformation (via Lambda) supports format conversion (e.g., JSON to Parquet).
  - Compression (GZIP, Snappy).
- **Exam Tip:** If the question asks to "store streaming data into S3/Redshift," think Firehose.

## 11. SQS vs SNS vs Kinesis (Comparison Cheat Sheet)

| Feature | SQS | SNS | Kinesis Data Streams |
|---|---|---|---|
| **Model** | Pull (Polling) | Push (Pub/Sub) | Pull (Real-time Streaming) |
| **Entity** | Queue | Topic | Stream (Shards) |
| **Consumption** | 1 consumer (usually) deletes msg after processing | Many subscribers receive copy | Multiple consumers can read same stream (Replay) |
| **Persistence** | Yes (Deleted on success) | No (Lost if not delivered*) | Yes (Expires after retention) |
| **Use Case** | Decoupling jobs, Batch processing | Notifications, Fan-out | Real-time Analytics, Big Data, Clickstreams |

**Note: SNS can utilize DLQs for failed deliveries, but doesn't persist history like Kinesis.*

## 12. Amazon MQ

- **What is it?** Managed Apache ActiveMQ and RabbitMQ service.
- **When to use (Exam Scenario):**
  - You are migrating an existing on-premise application to AWS.
  - The app uses standard protocols like **MQTT, AMQP, STOMP, OpenWire**.
  - You do **not** want to rewrite the application code to use SQS/SNS APIs.
- **Architecture:**
  - Runs on dedicated servers (not serverless like SQS/SNS).
  - Multi-AZ setup required for High Availability (Active/Standby).